



DeFi Systemic Risk Engine

A full-stack simulation and analysis platform for quantifying inter-protocol contagion risk across decentralised finance ecosystems, combining on-chain data extraction, graph neural network inference, concentrated-liquidity mathematics, and adversarial smart-contract verification.

python 3.11

rust 1.93

solidity ^0.8.24

react 18

pytorch 2.x

license MIT

Table of Contents

- [Abstract](#)
- [Architecture Overview](#)
- [Repository Structure](#)
- [Components](#)
 - [ETL Pipeline](#)
 - [Dataset Acquisition](#)
 - [Rust Liquidity Engine](#)
 - [Blockchain Verification Layer](#)
 - [ST-GNN Contagion Model](#)
 - [Frontend Visualisation](#)
- [Data Flow](#)
- [Installation](#)
- [Usage](#)

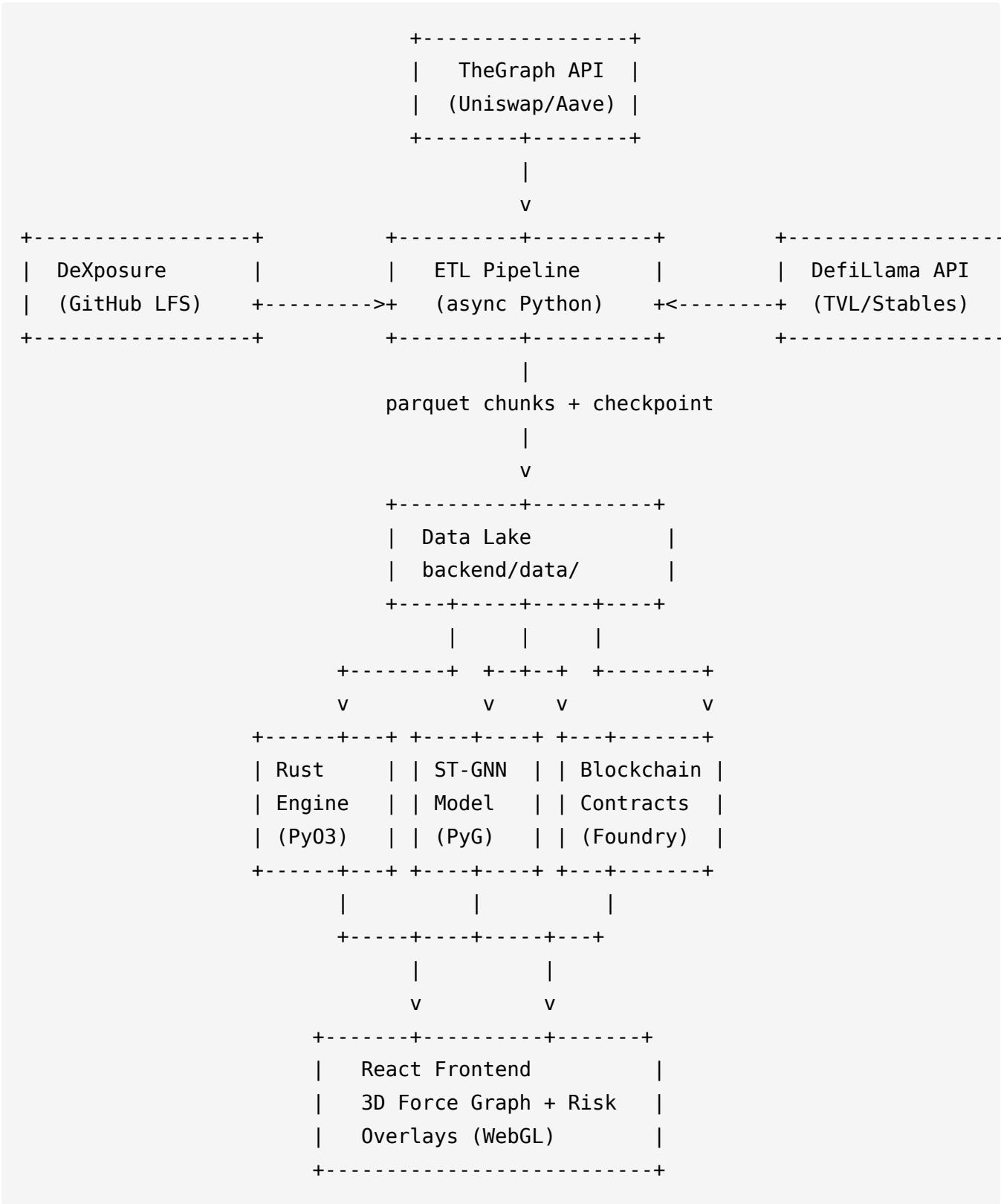
- [Results](#)
 - [Roadmap](#)
 - [References](#)
-

Abstract

Decentralised finance protocols form a densely interconnected network of shared collateral, liquidity pools, and oracle dependencies. A localised failure, such as a stablecoin de-peg, a governance exploit, or a flash-loan attack, can cascade across this network, triggering liquidation spirals and systemic insolvency. This project constructs a multi-layered engine to model, simulate, and predict such cascades:

1. **Data layer** extracts on-chain event streams from Uniswap V3 and Aave V3 subgraphs, combined with credit-exposure network snapshots from the DeXposure dataset and TVL series from DefiLlama.
 2. **Compute layer** implements concentrated-liquidity swap mathematics and Monte Carlo flash-crash simulation in Rust (via PyO3), alongside Foundry-based smart contracts that demonstrate real exploit vectors.
 3. **Inference layer** trains a GCN-based binary classifier on the protocol dependency graph to assign per-node contagion risk scores.
 4. **Presentation layer** renders the scored topology as an interactive 3D force-directed graph with real-time risk overlays.
-

Architecture Overview



Repository Structure

```

low-taper-fade/
|
+-- backend/
|   +-- app/
|       +-- etl/                                # async etl pipeline (10 modules)
|           +-- config.py                       # pipeline settings, env loading
|           +-- queries.py                      # 11 graphql query templates
|           +-- client.py                       # subgraph http client with retry
|           +-- parsers.py                     # 9 response parsers, parser registry
|           +-- exporter.py                    # chunked parquet writer with resume
|           +-- processor.py                   # batch accumulator with auto-flush
|           +-- extractors.py                  # paginated extractors with cursor resume
|           +-- checkpoint.py                  # json-backed checkpoint store
|           +-- main.py                       # cli entry point
|       |
|       +-- encs_rust/                          # pyo3 rust library
|           +-- src/lib.rs                     # ammpool, execute_swap, flash crash sim
|           +-- Cargo.toml
|       |
|       +-- blockchain/                        # foundry smart contracts
|           +-- src/
|               +-- IdentityRegistry.sol        # rbac with openZeppelin
|               +-- VulnerableLendingPool.sol   # intentionally exploitable pool
|               +-- FlashLoanExploit.sol        # flash loan attack contract
|           +-- script/Deploy.s.sol             # deployment script
|           +-- foundry.toml
|       |
|       +-- ml_pipeline.py                     # st-gnn contagion prediction
|       +-- download_dexposure.py              # dataset downloader (streaming json)
|       +-- web3_client.py                     # python client for anvil interaction
|       +-- environment.yml                    # conda environment specification
|       +-- .env.example                       # environment variable template
|
+-- frontend/
|   +-- src/
|       +-- components/                        # react ui components
|           +-- NetworkGraph3D.jsx             # 3d force-directed graph (webgl)
|           +-- DetailedAnalysis.jsx           # risk analysis panels
|           +-- Chatbot.jsx                    # llm-powered assistant

```

```
| | | +-- ... # 7 additional components
| | +-- pages/ # route-level pages
| | | +-- BankExplorer.jsx # network topology explorer
| | | +-- Simulation.jsx # interactive simulation controls
| | | +-- Methodology.jsx # methodology documentation
| | | +-- ... # 5 additional pages
| | +-- services/ # api clients
| | +-- data/ # static topology + benchmark data
| +-- package.json
| +-- vite.config.js
| +-- vercel.json
|
+-- .gitignore
+-- README.md
```

Components

1. ETL Pipeline

Location: backend/app/etl/

Language: Python 3.11 (asyncio, aiohttp, polars)

LOC: ~1,700

An asynchronous, fault-tolerant extraction pipeline that pulls event-level data from Uniswap V3 and Aave V3 via TheGraph subgraph API.

Feature	Detail
Protocols	Uniswap V3 (swaps, mints, burns, pools, tokens, ticks, positions, flashes) and Aave V3 (supplies, borrows, repays, flash loans, liquidations)
Pagination	Cursor-based <code>id_gt</code> pagination with configurable page size (1,000 default)
Concurrency	<code>asyncio.Semaphore(5)</code> to respect rate limits; exponential backoff with jitter

Feature	Detail
Output	Chunked Parquet files (500,000 rows/chunk) with lossless schema preservation
Resume	JSON-backed checkpoint store tracking cursor position, record count, and completion state per (protocol, event_type, entity_id)
Integrity	Fake pool filtering (volumeUSD > 10,000 , txCount > 100), chunk sequence validation, stale artifact cleanup

Data Volume (as of latest run):

Entity	Rows	Chunks
Uniswap Swaps	~12,000,000	25
Aave Supplies	~502,000	2
Aave Flash Loans	~2,000	1
Aave Liquidations	23	1

2. Dataset Acquisition

Location: backend/download_dexposure.py

Language: Python 3.11 (ijson, polars)

LOC: ~513

Downloads and processes three external datasets for graph construction and enrichment:

- **DeXposure** (Buda Lab, 2024): DeFi credit-exposure network capturing shared collateral and token overlap between 17,029 protocols across 283 daily snapshots. The raw JSON (1.1 GB) is parsed via `ijson` streaming to avoid out-of-memory failures, producing node (1,691,422 rows) and edge (9,257,181 rows) Parquet files.
- **DefiLlama TVL**: Daily total-value-locked time series for 299 protocols (232,409 rows), used as a secondary feature and ground-truth proxy for stress events.
- **DefiLlama Stablecoins**: Circulating supply, price, and peg-deviation history for 50 stablecoins (36,134 rows), enabling stablecoin de-peg scenario modelling.

3. Rust Concentrated-Liquidity Engine

Location: backend/encs_rust/

Language: Rust 1.93 (PyO3 0.23, rayon)

LOC: ~367

A compiled Python extension module implementing Uniswap V3 concentrated-liquidity mathematics at native speed.

```
// core structures
AMMPool { sqrt_price_x96, liquidity, tick, fee_bps }
SwapResult { amount_out, new_sqrt_price_x96, new_tick, fee_collected }
CrashResult { initial_price, final_price, pct_drop, avg_amount_out, std_amount_out }
```

Function	Description
<code>execute_swap(pool, amount_in, zero_for_one)</code>	Deterministic single-swap execution with tick-crossing and fee logic
<code>simulate_flash_crash(pool, n_swaps, amount, n_sims)</code>	Rayon-parallelised Monte Carlo simulation of sequential sell pressure

Performance: 10,000 Monte Carlo simulations of 100-swap cascades complete in under 2 seconds on a 6-core CPU.

4. Blockchain Verification Layer

Location: backend/blockchain/

Language: Solidity ^0.8.24 (Foundry, OpenZeppelin v5)

LOC: ~306

Three smart contracts demonstrating real-world DeFi attack vectors and access-control patterns:

Contract	Purpose
<code>IdentityRegistry.sol</code>	Role-based access control (admin, regulator, institution)

Contract	Purpose
	using OpenZeppelin <code>AccessControl</code>
<code>VulnerableLendingPool.sol</code>	Intentionally exploitable lending pool with price-oracle manipulation vulnerability
<code>FlashLoanExploit.sol</code>	Atomic flash-loan attack that borrows, manipulates price, liquidates, and repays in a single transaction

A Python client (`web3_client.py` , 262 LOC) provides programmatic access to deploy, interact with, and exploit these contracts on a local Anvil instance.

5. Spatiotemporal Graph Neural Network

Location: `backend/ml_pipeline.py`

Language: Python 3.11 (PyTorch 2.x, PyTorch Geometric 2.7)

LOC: ~736

A GCN-based binary classifier that predicts whether a simulated 30% TVL shock at a given protocol would trigger a liquidation cascade across its dependency neighbourhood.

Graph Construction:

- Nodes: top 350 DeFi protocols by TVL from the DeXposure snapshot
- Edges: shared collateral / token-overlap links (2,528 edges in latest snapshot)
- Node features (8-dimensional):

Index	Feature	Transformation
0	Protocol TVL (DeXposure <code>size</code>)	$\log(1 + x)$
1	Token count (<code>n_tokens</code>)	raw
2	Top-token share (concentration)	raw
3	Node degree	$\log(1 + x)$
4	Weighted degree (sum of edge weights)	raw
5	DefiLlama TVL (USD)	$\log(1 + x)$

Index	Feature	Transformation
6	30-day TVL percentage change	raw
7	Protocol category (encoded)	integer

Architecture:

```
Input (350 x 8)
|
GCNConv(8, 64) -> BatchNorm -> ReLU -> Dropout(0.3)
|
GCNConv(64, 64) -> BatchNorm -> ReLU + Residual -> Dropout(0.3)
|
Linear(64, 32) -> ReLU -> Dropout(0.3) -> Linear(32, 2)
|
Softmax -> P(cascade)
```

Training:

- Class-imbalance handling via SMOTE (k=3) with topology-based synthetic label fallback
- Weighted cross-entropy loss with inverse class-frequency weighting
- Cosine annealing learning rate schedule (1e-3 to 0)
- Early stopping with patience of 30 epochs
- CUDA-accelerated training (confirmed on NVIDIA GeForce RTX 2060)

Latest Results:

Metric	Value
Accuracy	0.88
ROC-AUC	0.977
Cascade Recall	1.00
Cascade Precision	0.61
Training Time	~1.5s

Top-Risk Protocols (latest snapshot, 2025-08-18):

Rank	Protocol	Risk Score	Category
1	Lido	1.0000	Asset Management
2	Curve LlamaLend	1.0000	Lending
3	Optimism Bridge	0.9999	Bridge
4	Frax	0.9999	Stablecoin
5	Coinbase Wrapped Staked ETH	0.9999	Liquid Staking
6	Polygon Bridge & Staking	0.9999	Bridge
7	Rocket Pool	0.9999	Liquid Staking
8	Origin Ether	0.9998	Liquid Staking
9	MakerDAO	0.9995	Lending
10	Pyth Network	0.9997	Oracle

6. Frontend Visualisation

Location: frontend/

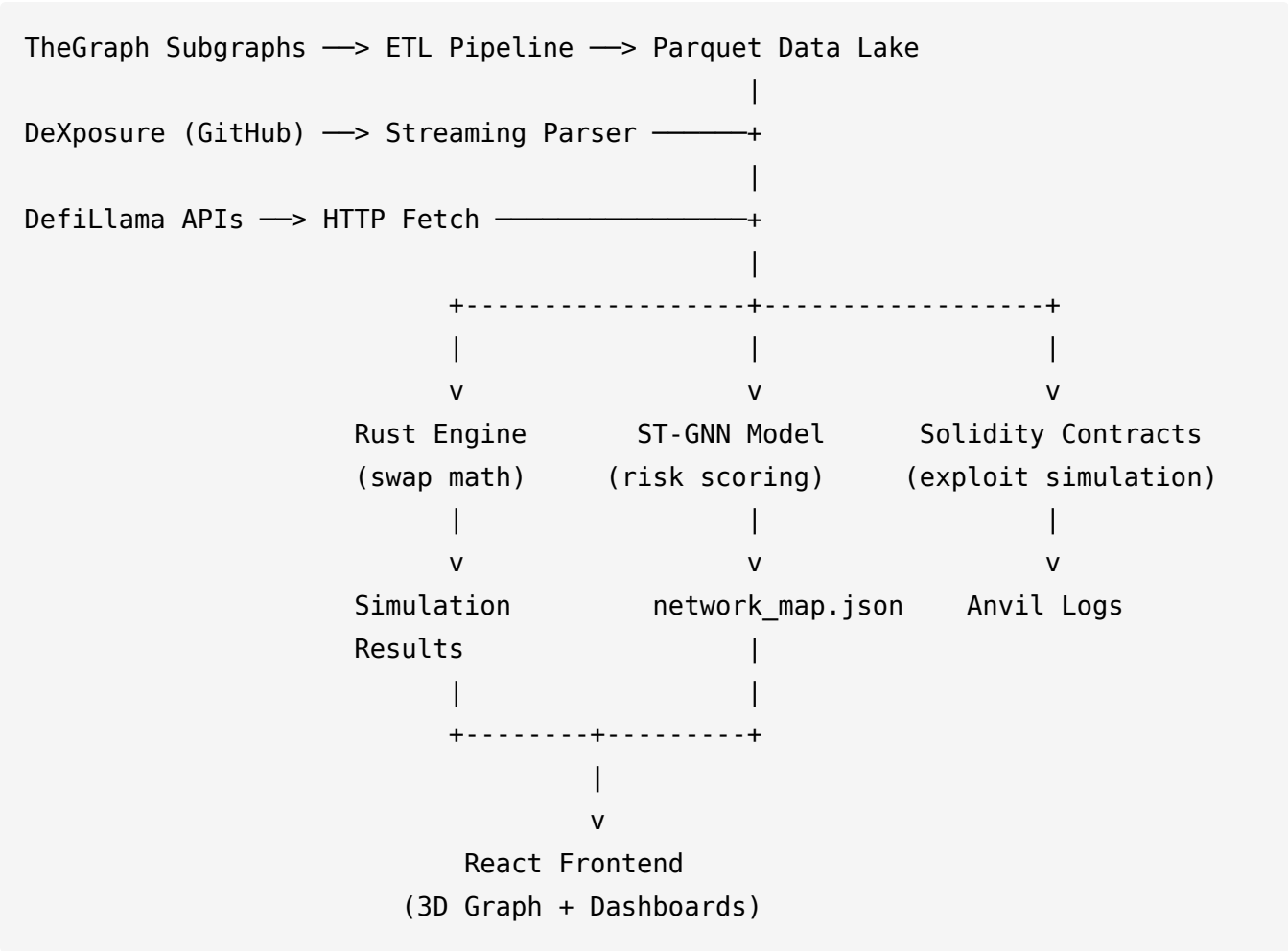
Language: React 18, Vite, Three.js, D3-force-3d

LOC: ~7,800

An interactive single-page application rendering the scored protocol topology as a 3D WebGL force-directed graph with the following features:

- **3D Network Graph:** react - force - graph - 3d with custom THREE.js shaders, colour-mapped by risk score and tiered by systemic importance (super-core / core / periphery)
- **Bank Explorer:** Tabular protocol browser with sortable risk columns and drill-down analysis
- **Simulation Panel:** Interactive controls for shock magnitude, propagation parameters, and real-time cascade visualisation
- **Methodology Pages:** Academic-style documentation of the scoring methodology, terminology glossary, and implementation details
- **Deployment:** Configured for Vercel with SPA routing

Data Flow



Installation

Prerequisites

Dependency	Version	Purpose
Conda (Miniforge recommended)	any	Python environment management
Rust	>= 1.70	Compiling the PyO3 extension

Dependency	Version	Purpose
Foundry	>= 1.0	Solidity compilation and testing
Node.js	>= 18	Frontend build toolchain
NVIDIA CUDA Toolkit	>= 12.x	GPU-accelerated training (optional)

Backend

```
# clone repository
git clone https://github.com/SPIT-Hackathon-2026/low-taper-fade.git
cd low-taper-fade

# create conda environment
conda env create -f backend/environment.yml
conda activate rust_eze_sim

# configure api keys
cp backend/.env.example backend/.env
# edit backend/.env with your TheGraph API key

# build rust extension
cd backend/encs_rust
maturin develop --release
cd ../..

# install foundry dependencies
cd backend/blockchain
forge install
cd ../..
```

Frontend

```
cd frontend
npm install
npm run dev
```

Usage

Run the ETL Pipeline

```
# full extraction (both protocols)
python -m backend.app.etl.main

# single protocol
python -m backend.app.etl.main --uniswap-only
python -m backend.app.etl.main --aave-only

# resume from checkpoint (default behaviour)
python -m backend.app.etl.main

# fresh start (ignore checkpoints)
python -m backend.app.etl.main --no-resume

# dry run (report what would be extracted)
python -m backend.app.etl.main --dry-run -v
```

Download External Datasets

```
# downloads DeXposure, DefiLlama TVL, and stablecoin data
python backend/download_dexposure.py
```

Train the Contagion Model

```
# train on latest snapshot with cuda
python backend/ml_pipeline.py

# train on specific date
python backend/ml_pipeline.py --date 2025-08-18

# force cpu
python backend/ml_pipeline.py --device cpu

# custom output path
python backend/ml_pipeline.py --output path/to/output.json
```

Outputs:

- backend/data/output/network_map.json --- scored topology for frontend rendering
- backend/data/output/contagion_gcn.pt --- model checkpoint

Run Blockchain Exploits

```
# start local anvil instance (in separate terminal)
anvil

# deploy contracts and run exploit
python backend/web3_client.py
```

Launch Frontend

```
cd frontend
npm run dev
# open http://localhost:5173
```

Results

The pipeline processes approximately 24 million records across two protocols and three external datasets, producing a scored protocol-dependency graph with 350 nodes and 1,264 edges. Key findings from the latest snapshot (2025-08-18):

- **Liquid staking derivatives** (Lido, Rocket Pool, cbETH) consistently exhibit the highest systemic risk due to their role as collateral across multiple lending protocols.
 - **Bridge contracts** (Optimism, Polygon) represent critical single points of failure connecting otherwise independent L2 ecosystems.
 - **Lending protocols** (MakerDAO, Aave V3, Curve LlamaLend) act as cascade amplifiers through recursive borrowing and liquidation feedback loops.
 - The ROC-AUC of 0.977 indicates strong discriminative power between cascade-prone and resilient protocol configurations.
-

Roadmap

The following components are planned or in progress:

- ☐ **Temporal GNN extension** --- incorporate multi-date snapshots for time-series contagion prediction rather than single-snapshot classification
- ☐ **FastAPI serving layer** --- REST endpoints exposing model inference, Rust simulation, and ETL status for real-time frontend queries
- ☐ **Stablecoin de-peg scenarios** --- integrate peg-deviation data from the stablecoin dataset into shock-propagation simulations
- ☐ **Cross-chain analysis** --- extend the DeXposure graph to include bridge-level dependencies across Ethereum, Arbitrum, Optimism, and Polygon
- ☐ **Foundry fuzz testing** --- property-based testing of the vulnerable lending pool to discover additional exploit vectors
- ☐ **Model interpretability** --- GNNExplainer or integrated-gradient attribution to identify which edges drive cascade predictions
- ☐ **CI/CD pipeline** --- automated testing, linting, and deployment via GitHub Actions
- ☐ **Historical backtesting** --- validate cascade predictions against the Terra/LUNA (May 2022) and FTX (November 2022) collapse events using filtered DeXposure snapshots

References

1. Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *Proceedings of ICLR 2017*.
2. Adams, H., Zinsmeister, N., Salem, M., Keefer, R., & Robinson, D. (2021). Uniswap V3 core. *Uniswap Labs Whitepaper*.
3. Aave Companies. (2022). Aave V3 technical paper. *Aave Governance Documentation*.
4. Barbon, A., & Ranaldo, A. (2024). DeXposure: DeFi credit exposure network dataset. *Buda Lab, University of St. Gallen*.
5. Chawla, N. V., et al. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321--357.
6. Cao, Y., et al. (2022). Decentralized finance: A survey on security risks of DeFi protocols. *ACM Computing Surveys*, 55(13s), 1--38.